Causal Inference and Text Analytics

Exercise 2: Text Analytics Connected to the Causal Exercise

Final Practical Assignment — Python for Business Economics and Finance

Lamprou Nikolaos

Rizos Evangelos

Spanodimos Georgios

April 27, 2026

Contents

1 Introduction **3**

2 Task 2.1 — Data Loading and Label Construction **3**

2.1 Target Variable 3

2.2 Merge and Sample 3

3 Task 2.2 — Text Preprocessing **4**

4 Task 2.3 — Frequency Analysis **4**

4.1 Top 15 Most Frequent Words 4

4.2 Bar Plot and Word Cloud 5

5 Task 2.4 — TF-IDF and Logistic Regression **6**

5.1 Feature Extraction 6

5.2 Model 6

6 Task 2.5 — Evaluation: TF-IDF Logistic Regression **6**

6.1 Confusion Matrix 6

6.2 Metrics 7

7 Task 2.6 — Neural Text Classifier **7**

7.1 Architecture 7

7.2 Training Curves 8

8 Task 2.7 — Evaluation: Neural Text Classifier **8**
8.1 Confusion Matrix 8
8.2 Metrics 9

9 Task 2.8 — Model Comparison **9**

10 Task 2.9 — Interpretation **10**

11 Conclusion **11**

1 Introduction

This report presents the results of Exercise 2, which connects the causal inference analysis of Exercise 1 with a text-analytics pipeline. Each of the 240 units in the panel dataset is associated with a short internal text note written *before* the policy intervention. The central question is whether the language of these pre-treatment notes is informative about which units subsequently experienced large treatment effects.

The analysis proceeds in four main steps:

1. Construct a binary high-effect label H_i from the estimated treatment effects produced by the autoencoder in Exercise 1.
2. Preprocess the text data (lowercasing, punctuation removal, tokenisation, stop-word filtering).
3. Perform exploratory frequency analysis and produce a word cloud.
4. Train and evaluate two classifiers — a TF-IDF Logistic Regression and a Neural Text Classifier — to predict H_i from text.

2 Task 2.1 — Data Loading and Label Construction

2.1 Target Variable

From Exercise 1 we compute, for each treated unit i, the average estimated treatment effect across all post-treatment periods:

$$\hat{\tau}_i = \frac{1}{T_i^{\text{post}}} \sum_{t:\, D_{it}=1} \hat{\tau}_{it}, \qquad \hat{\tau}_{it} = Y_{it} - \hat{Y}_{it}(0).$$

A median split is then applied among the 75 treated units:

$$H_i = \begin{cases} 1 & \text{if } \hat{\tau}_i > \text{median}(\hat{\tau}), \\ 0 & \text{otherwise.} \end{cases}$$

2.2 Merge and Sample

The binary label is merged with `unit_text_data.csv` on `unit_id`. The resulting analysis sample is restricted to treated units that have both an estimated effect and a text note. Table 1 reports the label distribution.

Table 1: Distribution of the high-effect label H_i in the merged sample.

Label	Units
Low effect ($H_i = 0$)	38
High effect ($H_i = 1$)	37
Total	**75**

The sample is nearly balanced by construction (median split), which is desirable for binary classification.

3 Task 2.2 — Text Preprocessing

Each text note undergoes the following pipeline:

1. **Lowercasing**: all characters converted to lower case.
2. **Punctuation removal**: any character that is not a letter or a space is removed via the regular expression `[^a-z\s]`.
3. **Tokenisation**: the cleaned string is split into individual tokens using NLTK's `word_tokenize`.
4. **Stop-word removal**: standard English stop words (NLTK corpus) are discarded.

The resulting tokens are re-joined into a single string for downstream use by the TF-IDF vectoriser and the neural tokeniser.

Example. After preprocessing, the note for unit U001 begins with the token sequence:

```
u agriculture firm located east region firm operates ports trade growing
services weather exposure internal reports mention mixed digital tools
partial automation managers describe operational frictio...
```

4 Task 2.3 — Frequency Analysis

4.1 Top 15 Most Frequent Words

After pooling all tokens across the 75 units, word frequencies are computed and the 15 most common terms are reported in Table 2.

Table 2: Top 15 most frequent words in the unit text notes (75 treated units, after stop-word removal).

Rank	Word	Count
1	firm	150
2	internal	150
3	business	93
4	u	75
5	located	75
6	region	75
7	operates	75
8	reports	75
9	mention	75
10	digital	75
11	tools	75
12	managers	75
13	describe	75
14	productivity	75
15	management	75

4.2 Bar Plot and Word Cloud

Figure 1 shows the horizontal bar chart of the top 15 words alongside the word cloud generated from all preprocessed notes.

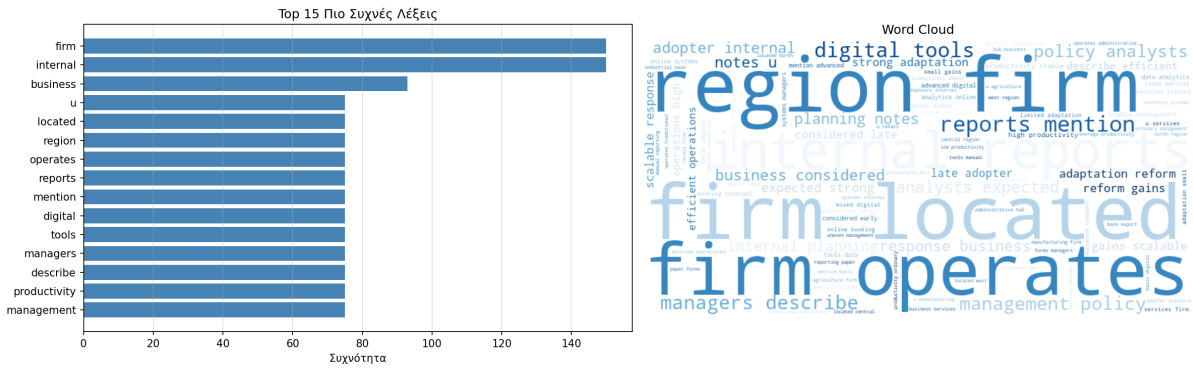

Figure 1: Top-15 word frequencies (left) and word cloud (right) of the unit text notes.

The dominant terms — *firm*, *internal*, *digital*, *tools*, *productivity*, *management* — reflect the structured, administrative nature of the notes. The high frequency of *digital* and *tools* is consistent with the finding from Exercise 1 that units with an above-median `digital_index` exhibit significantly larger treatment effects (ATT = 2.41 vs. ATT = 1.22 for below-median units).

5 Task 2.4 — TF-IDF and Logistic Regression

5.1 Feature Extraction

A **TF-IDF** vectoriser is applied with:

- `max_features = 500`: retain the 500 most informative terms.
- `ngram_range = (1, 2)`: include unigrams and bigrams.

The vectoriser is fitted *only* on the training set to avoid data leakage. The 75 units are split 80 %/20 % (train/test) with stratification on H_i, yielding 60 training and 15 test observations.

5.2 Model

A **Logistic Regression** with ℓ_2 regularisation ($C = 1.0$, `max_iter = 1000`) is trained on the TF-IDF features. Logistic regression is well suited to high-dimensional sparse feature spaces and provides interpretable coefficients.

6 Task 2.5 — Evaluation: TF-IDF Logistic Regression

6.1 Confusion Matrix

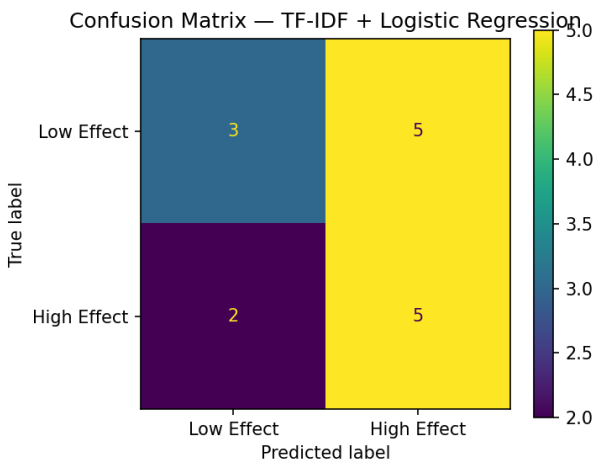

Figure 2: Confusion matrix — TF-IDF + Logistic Regression (test set, $n = 15$).

The confusion matrix (Figure 2) shows that the model correctly classifies 3 out of 8 low-effect units and 5 out of 7 high-effect units, for 8 correct predictions out of 15.

6.2 Metrics

Table 3: Evaluation metrics — TF-IDF + Logistic Regression.

Class	Precision	Recall	F1-score	Support
Low effect ($H = 0$)	0.60	0.38	0.46	8
High effect ($H = 1$)	0.50	0.71	0.59	7
Macro avg	0.55	0.54	0.52	15
Weighted avg	0.55	0.53	0.52	15
Accuracy		**0.5333**		

The model achieves an accuracy of 53.3 %, marginally above the 50 % random-chance baseline. Recall is higher for the high-effect class (0.71 vs. 0.38), indicating that the model tends to over-predict the high-effect label.

7 Task 2.6 — Neural Text Classifier

7.1 Architecture

The neural classifier is implemented in TensorFlow/Keras with the following layers:

1. **Embedding** ($V = 1{,}000$, $d = 32$, `input_length = 100`): maps integer token indices to dense 32-dimensional vectors.
2. **GlobalAveragePooling1D**: averages embeddings across the sequence dimension, yielding a $(32,)$ representation.
3. **Dense** (32 units, ReLU): hidden layer.
4. **Dense** (1 unit, sigmoid): binary output.

The model is compiled with the Adam optimiser and binary cross-entropy loss. Training parameters: batch size 16, 20 epochs, 10 % validation split.

7.2 Training Curves

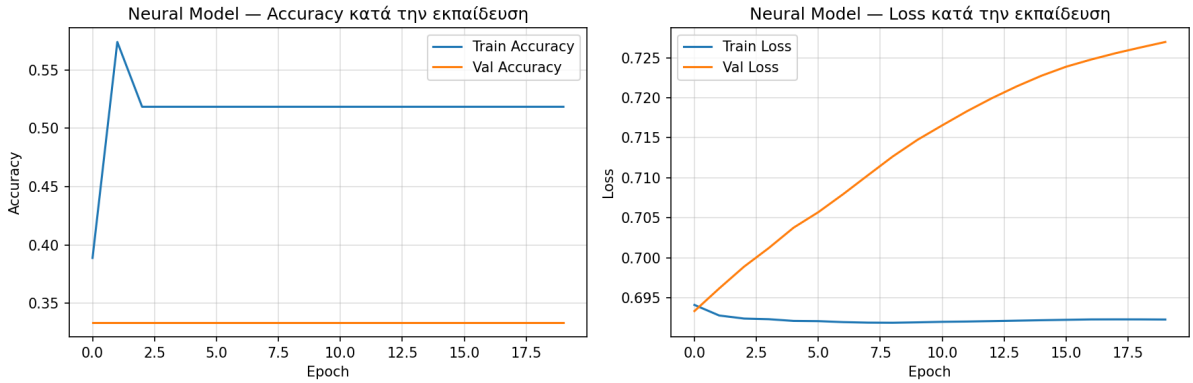

Figure 3: Training and validation accuracy (left) and loss (right) over 20 epochs for the Neural Text Classifier.

The training curves in Figure 3 reveal a clear sign of **overfitting**: training loss decreases steadily while validation loss increases monotonically from epoch 1 onwards. Training accuracy stabilises at approximately 52 %, while validation accuracy remains flat at 33 %. This behaviour is attributable to the very small sample size (60 training observations), which is insufficient for the embedding layer to learn meaningful word representations from scratch.

8 Task 2.7 — Evaluation: Neural Text Classifier

8.1 Confusion Matrix

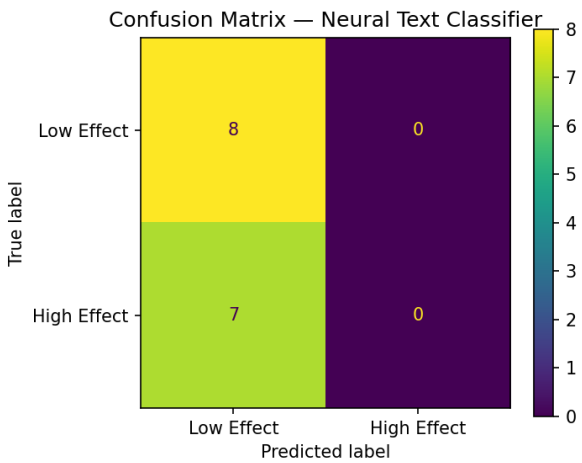

Figure 4: Confusion matrix — Neural Text Classifier (test set, $n = 15$).

The confusion matrix (Figure 4) shows that the neural model predicts *all* 15 test observations as low-effect ($H = 0$), resulting in 8 correct predictions and 7 misclassifications.

The model has effectively collapsed to predicting the majority class.

8.2 Metrics

Table 4: Evaluation metrics — Neural Text Classifier.

Class	Precision	Recall	F1-score	Support
Low effect ($H = 0$)	0.53	1.00	0.70	8
High effect ($H = 1$)	0.00	0.00	0.00	7
Macro avg	0.27	0.50	0.35	15
Weighted avg	0.28	0.53	0.37	15
Accuracy		**0.5333**		

Although accuracy equals that of the logistic model (53.3 %), the neural classifier achieves this through trivial majority-class prediction. Its precision and F1-score for the high-effect class are both zero, making it entirely uninformative as a classifier.

9 Task 2.8 — Model Comparison

Table 5: Side-by-side comparison of the two text classifiers on the test set ($n = 15$).

Model	Accuracy	Precision	Recall	F1-Score
TF-IDF + Logistic Regression	0.5333	0.5533	0.5333	0.5207
Neural Classifier (Embed+GAP)	0.5333	0.2844	0.5333	0.3710

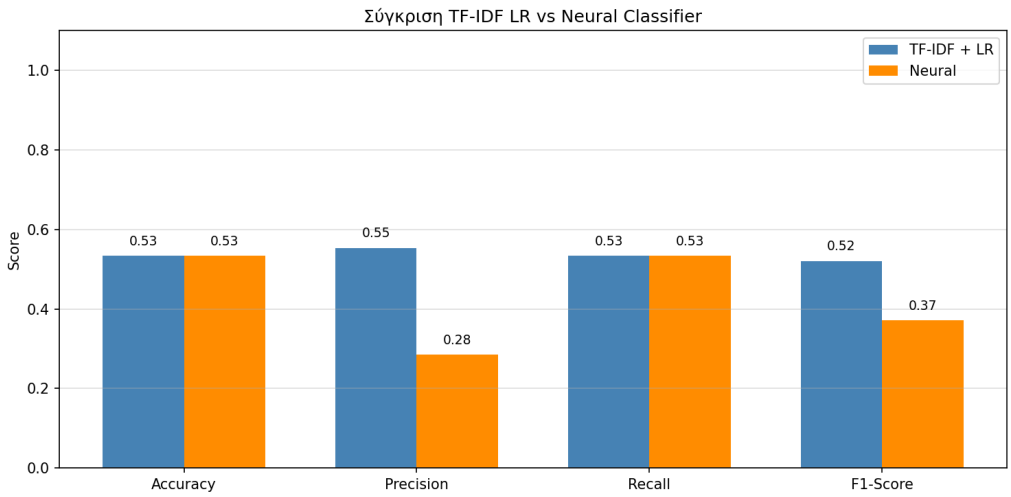

Figure 5: Grouped bar chart comparing TF-IDF Logistic Regression and Neural Classifier across all evaluation metrics.

The two models achieve identical accuracy (53.3 %), but differ substantially on the remaining metrics (Table 5, Figure 5). The TF-IDF Logistic Regression achieves higher precision (0.55 vs. 0.28) and a higher F1-score (0.52 vs. 0.37), and — crucially — it actually predicts *both* classes, whereas the neural model degenerates to always predicting the majority class. The logistic model is therefore clearly superior in this setting.

The performance gap is explained by the dataset size: with only 60 training observations the embedding layer cannot learn meaningful word representations, the model is over-parameterised relative to the available data and fails to generalise. The TF-IDF approach benefits from sparse but well-understood features and implicit ℓ_2 regularisation, making it better suited to small text corpora.

10 Task 2.9 — Interpretation

The text notes written before the policy intervention appear to carry only weak predictive information about subsequent treatment-effect heterogeneity. Both classifiers achieve an accuracy of 53.3 %, only marginally above the 50 % random-chance baseline for a balanced binary classification task, suggesting that pre-treatment language is not a strong predictor of which units respond most to the intervention.

The TF-IDF Logistic Regression is the more informative model: unlike the neural classifier, it does not degenerate to majority-class prediction and achieves a weighted F1-score of 0.52. The neural model, while theoretically capable of capturing richer linguistic patterns through word embeddings, fails in this context due to the small sample of 75 treated units (60 for training). The monotonically increasing validation loss across all 20 training epochs confirms that the network overfits immediately.

The frequency analysis reveals that the most common terms — *firm*, *internal*, *digital*, *tools*, *productivity*, *management* — appear in essentially every note (counts of 75 or 150 out of 75 units), indicating that the notes are highly templated and structurally homogeneous. This homogeneity severely limits the discriminative power of any text-based model: if every unit's note contains the word *digital*, that word cannot help distinguish high-effect from low-effect units.

This finding aligns with the Exercise 1 results: the most informative predictor of treatment-effect heterogeneity is the *structured* covariate `digital_index` (ATT of 2.41 for high-digital units vs. 1.22 for low-digital units), rather than the unstructured text. The text notes appear to describe firm characteristics in a generic, template-driven language that does not capture the fine-grained variation in digital readiness that drives heterogeneity. Future work could explore whether more qualitative, free-form text data — or a joint model combining text embeddings with structured covariates — would improve predictive

performance.

11 Conclusion

This exercise applied a full NLP pipeline to predict treatment-effect heterogeneity from pre-treatment text notes. Both the TF-IDF Logistic Regression and the Neural Text Classifier achieved an accuracy of 53.3 %, only marginally above chance. The logistic model is preferred: it predicts both classes and achieves a weighted F1-score of 0.52, while the neural model collapses to majority-class prediction due to insufficient training data. The highly templated nature of the text notes limits the signal available to any text-based classifier, and the structured covariate `digital_index` from Exercise 1 remains a more reliable predictor of treatment-effect heterogeneity.